

Assignment Report

Title: **Compiler Principles**

The Forth Theory Homework

Name	林宏宇
Student ID	23320093
School	计算机学院
Major	计算机科学与技术
Instructor	李文军

[ABSTRACT]

After learning the lecture 4 of compiler principles[1], we got the homework at the end of the lecture.

There are some requirements for the homework:

- The file should be named as **“学号 _ 姓名 _ 第 i 次理论作业”**, where i is the number of the homework.
- The file should be in docx or **pdf** format.
- All the homework should be sent to the TA's email: **406648896@qq.com**
- The deadline for the homework is **24:00 on Sunday of the week** when the homework is assigned.

Keywords: Compiler Principles

Contents

1 Exercise 4.1 1
1.1 Problem 1
1.2 Solution 1

2 Exercise 4.2 4
2.1 Problem 4
2.2 Solution 4

3 Exercise 4.3 4
3.1 Problem 4
3.2 Solution 5

References 6

1 Exercise 4.1

1.1 Problem

Given the following grammar:

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

Tasks:

1) Eliminate left recursion in the grammar.
2) Draw the transition diagrams for the grammar
3) Write recursive-descent (predictive) procedures for the grammar.
4) Indicate the procedure call sequence for the input sentence "(a,(a,a))".

1.2 Solution

1.2.1 Eliminate left recursion

The nonterminal L is left-recursive. We rewrite L as a right-recursive list (as shown):

$$S \rightarrow (L) \mid a$$

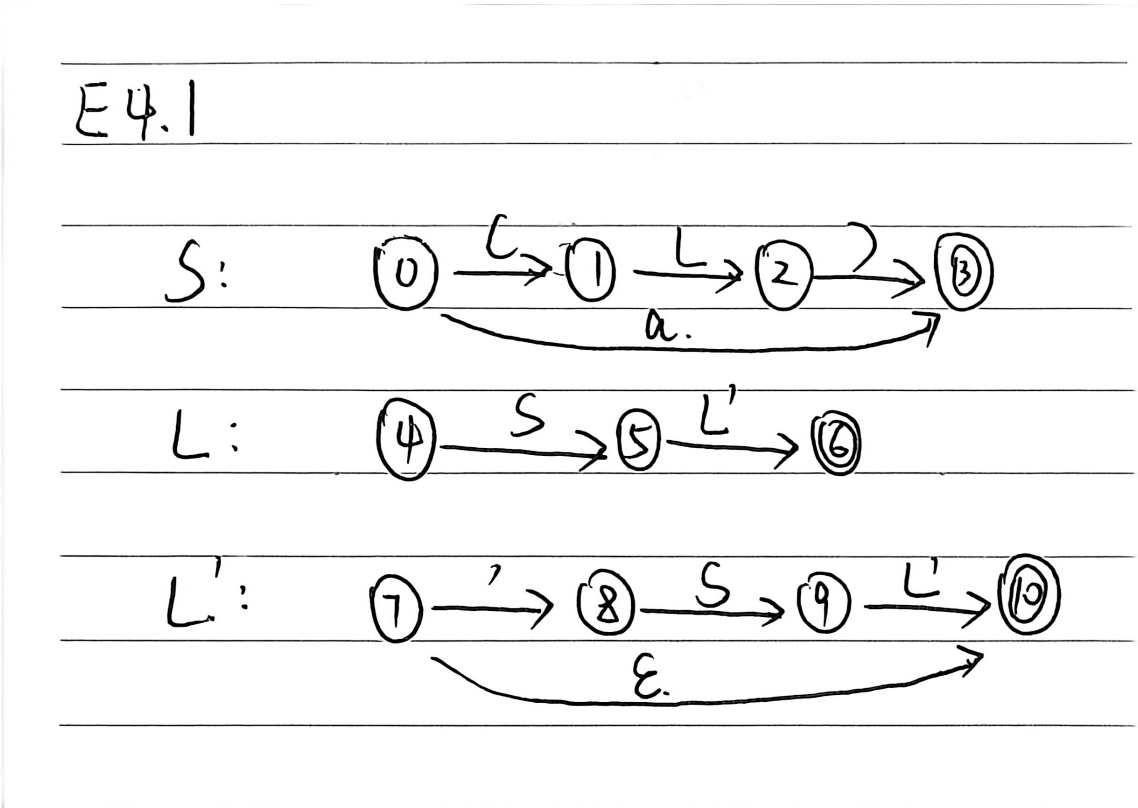
$$L \rightarrow SL'$$

$$L' \rightarrow,\ S\ L' \mid \epsilon$$

The start production for S remains $S \rightarrow (L) \mid a$.

1.2.2 Draw the transition diagrams for the grammar

The transition diagrams for the grammar are as follows:



1.2.3 Predictive recursive-descent procedures (pseudocode)

There are three procedures corresponding to the nonterminals S, L, and L'. Each procedure uses the lookahead token to decide which production to apply.

AS for S:

```
void S( ) {
    if ( lookahead == '(' ) {
        match( '(' );
        L( );
        match( ' )' );
    }
    else if ( lookahead == a ) {
        match( a );
    }
    else{
        error( );
    }
}
```

AS for L':

```
void L1( ) {
    if ( lookahead == ',' ) {
        match( ',' );
        S( );
        L1( );
    }
    else if ( lookahead in FOLLOW(L') ) {
    }
    else{
        error( );
    }
}
```

AS for L:

```
void L1( ) {
    S();
    L1();
}
```

1.2.4 Procedure call sequence for the input "(a,(a,a))"

Initial input: `( a , ( a , a ) )`, initial `lookahead='('`.

1) Call `S()`: match `'('`; now `lookahead='a'`.
2) Call `L()` to parse the list.
3) Call `S()`: match `a`; now `lookahead=','`.
4) Back in outer `L()`, see `','`: match it; now `lookahead='('`.
5) Call `S()`: match `'('`; now `lookahead='a'` (enter inner list).
6) Call inner `L()`.
7) Call `S()`: match `a`; now `lookahead=','`.
8) In inner `L()`, see `','`: match it and call `S()` for the next element.
9) Call `S()`: match `a`; now `lookahead=')'`.
10) In inner `L()`, `')'` is in FOLLOW(L), so `L'` -> ϵ (no further matches).
11) Return to inner `S()`: match closing `')'`; inner `S` completes; now `lookahead=','` (back to outer list).
12) Continue outer `L()`, then return to outer `S()` and match closing `')'`; input accepted.

2 Exercise 4.2

2.1 Problem

Consider the context-free grammar

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

Can you construct a predictive (LL(1)) parser for this grammar? Explain why or why not.

2.2 Solution

No, we cannot construct a predictive (LL(1)) parser for this grammar. The grammar is ambiguous because the same string can be derived in multiple ways. For example, the string ”ab” can be derived as follows:

$$S \rightarrow aSbS \rightarrow a\epsilon b\epsilon = ab$$

or

$$S \rightarrow bSaS \rightarrow b\epsilon a\epsilon = ba$$

3 Exercise 4.3

3.1 Problem

Compute the FIRST and FOLLOW for the start symbol of the following grammar:

$$S \rightarrow S\ S\ +\ \mid S\ S\ *\ \mid a$$

3.2 Solution

3.2.1 Eliminate Left Recursion

The given grammar contains left recursion. To eliminate it, we rewrite the grammar as follows:

$$S \rightarrow aS'$$

$$S' \rightarrow S + S' \mid S * S' \mid \epsilon$$

3.2.2 Left Factoring

To make the grammar suitable for predictive parsing, we perform left factoring:

$$S \rightarrow aS'$$

$$S' \rightarrow ST \mid \epsilon$$

$$T \rightarrow +S' \mid *S'$$

3.2.3 FIRST and FOLLOW Sets

The FIRST and FOLLOW sets for the start symbol S are computed as follows:

$$\text{FIRST}(S) = \{a\}$$

$$\text{FOLLOW}(S) = \{+, *, \$\}$$

References

[1] JUN LI W. Lecture03. lexical analysis[Z]. 2026.